# MaskIN

Use ChatGPT on private Indian documents without ChatGPT ever seeing the real details.

| | |
|---|---|
| Event | Hacktoberfest Hack Day 2026, NST Pune (NSTSDC), 24 to 25 Oct |
| Problem statement | #3: PII redaction for Indian documents and languages |
| Track | Track 1, Main Track: Best Open-Source AI Project |
| What we build | A Chrome extension for chatgpt.com, powered by Gemma 4 running locally on a Mac through Ollama |
| Open-source AI | Gemma 4 (open-weight model), run with Ollama (MIT). Our code: MIT License, public GitHub repo |
| Status of this doc | Final. The code base is built from this spec. No feature outside this doc gets built |

### Contents


1. The problem and what the slide asks for
2. The idea in one page
3. The user journey, every screen and popup in order
4. Who does what: regex, Gemma, code, ChatGPT
5. Tech stack, part by part
6. Data and privacy: what stays, what leaves
7. Files and folder structure
8. Setup on a Mac
9. Use cases we demo and testing
10. Build plan and team roles
11. Demo script and pitch
12. Judge questions and answers
13. Known limits (said honestly)
14. Glossary

# 1. The problem and what the slide asks for

> **Slide, PS #3.** Problem: people paste documents with names, IDs and account numbers into cloud chatbots. Who it helps: anyone using cloud AI with private documents. What to build: help people use online chat tools with private documents without exposing names, IDs or account numbers.

**What this means for us.** The user already uses ChatGPT and trusts it. We do not ask them to change that habit. We sit between the user and ChatGPT, like Grammarly sits inside the text box. Private details are swapped before they leave the laptop, and put back when the answer comes in.

**Why 'Indian' matters.** The slide says Indian documents and languages. So MaskIN understands Aadhaar, PAN, IFSC, UPI IDs, Indian phone numbers, PIN codes and bank account numbers, and handles names and addresses written in English, Hindi and Marathi (Devanagari script).

**What we hide and what we keep.** We hide what points to a person: names, addresses and ID or account numbers. We do not hide salary, loan amounts, prices or medical details. On their own these numbers do not point to anyone, and the user needs ChatGPT to see them to do calculations. Once the name and IDs are swapped, they belong to nobody.

**Example task.** A user has 10 KYC letters. They want ChatGPT to make a table of every name, address and phone number. With MaskIN, ChatGPT builds the table using fake names and numbers, and the user sees the table with the real values, on their own screen only.

# 2. The idea in one page

**One line:** MaskIN swaps your private details with realistic fakes before they reach ChatGPT, using Gemma running on your own Mac, and puts the real details back in ChatGPT's reply on your screen.

### The 8 steps (everyone in the team must know these by heart):

1. **Type, paste, or drop a document** (PDF, scanned PDF or photo) while using ChatGPT as usual.
2. **Click Mask** (the MaskIN button next to Send) or press **Option+Shift+M**.
3. **Detect on the laptop:** regex finds Indian IDs and numbers, Gemma finds names and addresses.
4. **Review:** a popup lists everything found. The user can untick items or add missed ones.
5. **Swap:** each item is replaced by a fake. The mapping (fake to real) is saved in the browser on this laptop.
6. **Send:** the user hits Send. ChatGPT only ever receives the fake version.
7. **Restore:** when ChatGPT replies, MaskIN swaps fakes back to real values, on screen only.
8. **Badge:** the reply shows how many values were restored and warns if something looks changed.

### Three lines to remember for the judges:

- Regex does exact formats, Gemma does understanding, plain code does storing and restoring.
- Everything sensitive stays on the laptop. Gemma runs in Ollama on localhost, the mapping is in local browser storage.
- ChatGPT only ever sees fakes. Real values come back only on the user's own screen.

### Extras in scope (decided, final):

- **Send guard:** if the user forgets to click Mask and presses Send with private details in the box, MaskIN stops the send and asks.
- **Copy real:** each restored reply gets a 'Copy real' button that copies the text with the real values.
- **Document input:** drop a PDF (text or scanned) or a photo (JPG, PNG) into the side panel. Text PDFs are read directly, scans and photos are read with OCR, all on the laptop. The text then goes through the same pipeline.
- **Regex-only mode:** if Ollama is not running, MaskIN still works for IDs and numbers, and says so clearly.

# 3. The user journey, every screen and popup in order

This is exactly what the user sees, from install to answer. Each item names the part of the extension that does it.

### A. One-time setup

| # | What the user sees | What happens behind it | Part |
|---|---|---|---|
| A1 | Installs Ollama, downloads Gemma, loads MaskIN in Chrome | Section 8 has the exact commands | Setup |
| A2 | Clicks the MaskIN icon. The **side panel** opens with a **Status card**: 'Local AI: Connected, gemma4:e4b' (green) or 'Local AI: Offline, regex-only mode' (amber) with a 'How to fix' link | Background worker calls GET `localhost:11434/api/tags` and checks the chosen model is installed | Side panel + background |
| A3 | Side panel **Settings**: model (e4b or e2b), Send guard on/off, Fake style (Realistic fakes or Tags like [NAME_1]) | Saved in chrome.storage.local | Side panel |

### B. Masking a message

| # | What the user sees | What happens behind it | Part |
|---|---|---|---|
| B1 | On chatgpt.com, a small **Mask** button sits next to ChatGPT's Send button. The user types or pastes as usual | Content script finds the input box using selectors from config.js and adds the button | Content script |
| B2 | User clicks Mask (or Option+Shift+M). A **toast** shows: 'Scanning on your laptop...' | Text is sent to the background worker. The pattern check (regex) runs instantly. Gemma runs on the text in chunks of about 1,500 characters | Background + Ollama |
| B3 | The **Review popup** opens over the page. Items are grouped by type with a colour chip, a one-line reason, and a preview: 'Ramesh Kumar → Suresh Patil'. Each has a tick box (on by default) | Regex items and Gemma items are merged. Overlaps are removed (longer match wins). Fakes are generated | Content script (popup in Shadow DOM) |
| B4 | User can untick an item, or type a missed word into '+ Add item'. Buttons: **Apply mask** and **Cancel** | Added items are treated as type 'Custom' and get a tag-style or fake value | Content script |
| B5 | After Apply, the text in ChatGPT's box is replaced by the masked text. Toast: '7 details masked. Nothing has been sent yet.' | Mapping saved for this chat. Text inserted with an input event so ChatGPT registers it | Content script + storage |
| B6 | User presses Send normally | ChatGPT receives only the masked text | ChatGPT |

### C. Send guard (only if the user forgot to mask)

| # | What the user sees | What happens behind it | Part |
|---|---|---|---|
| C1 | User presses Enter or clicks Send without masking. A **Send guard popup**: 'MaskIN found 3 private details (1 Aadhaar, 2 phone numbers). Mask now / Send anyway / Cancel' | MaskIN catches the Send before ChatGPT does and runs only the quick pattern check for IDs and numbers (it takes milliseconds). Gemma is not used here, so Send never feels slow. If nothing is found, the send goes through untouched | Content script |
| C2 | 'Mask now' opens the Review popup (B3). 'Send anyway' sends as is. 'Cancel' returns to editing | | Content script |

## D. The reply comes back

| # | What the user sees | What happens behind it | Part |
|---|---|---|---|
| D1 | ChatGPT writes its answer using the fakes. While it is still typing, nothing changes | A MutationObserver watches the chat. MaskIN waits until the reply has not changed for 1.2 seconds | Content script |
| D2 | The fakes in the reply turn into the real values, with a light dotted underline. Hovering shows 'Restored locally by MaskIN'. The user's own sent message also shows real values again | Only text nodes inside message bubbles are changed (no HTML rewriting). Longest values are replaced first. First names map too (Suresh → Ramesh) | Content script |
| D3 | Under the reply, a **badge**: 'MaskIN: 12 restored'. Two small buttons: **Copy real** and **Show what ChatGPT saw** (toggles the fake view) | Copy real writes the restored text to the clipboard. The toggle swaps the bubble text back and forth | Content script |
| D4 | If the reply contains an ID-like value that MaskIN never created (for example ChatGPT changed one digit of a fake PAN), the badge turns amber: '1 value looks changed, please check' | The regex detectors run on the reply and compare against the mapping | Content script |

## E. Dropping a PDF

| # | What the user sees | What happens behind it | Part |
|---|---|---|---|
| E1 | In the side panel, the user drops a PDF, JPG or PNG into the **Document** area | Text PDF: pdf.js reads the text directly. The file is never uploaded | Side panel + pdf.js |
| E2 | For a scanned PDF or a photo, a progress line shows: 'Reading scanned page 2 of 5 on your laptop...' | If pdf.js finds almost no text, each page is drawn as an image and read by Tesseract.js OCR (English, Hindi, Marathi), fully offline. Limits: 20 pages for text PDFs, 10 pages for scans | Side panel + pdf.js + Tesseract.js |
| E3 | The same Review list appears inside the side panel. Button: **Insert masked text into ChatGPT** | Same pipeline as B2 to B5. The masked text is placed in ChatGPT's box | Side panel + content script |
| E4 | If OCR finds very little text (blurry photo): 'Could not read this clearly. Try a sharper photo.' | OCR confidence too low | Side panel |

## F. Managing saved data

| # | What the user sees | What happens behind it | Part |
|---|---|---|---|
| F1 | Side panel **This chat** section: the list of masked items for the open chat (type, fake, real hidden behind a 'show' click) | Reads the mapping for the current chat | Side panel |
| F2 | Buttons: **Clear this chat** and **Clear all MaskIN data**, each with a confirm step | Deletes keys from chrome.storage.local | Side panel |

# 4. Who does what: regex, Gemma, code, ChatGPT

This table is the heart of every technical answer in Q&A.

| Job | Done by | Why this choice |
|---|---|---|
| Aadhaar, PAN, IFSC, phone, email, UPI, PIN, account number, date of birth | Regex + validation rules | Fixed formats. Must be exact and instant. AI models are unreliable with digits |
| Check that a 12-digit number is really an Aadhaar | Verhoeff checksum (code) | Aadhaar's last digit is calculated from the first 11. This stops order IDs and random numbers from being flagged |
| Names, addresses | Gemma 4, local | No fixed format, in English or Devanagari. Needs language understanding |
| Reading scans and photos | Tesseract.js OCR, local | Turns an image into text so the same detection can run on it |
| Style of a person's name (language, gender) | Gemma 4, local | So a Marathi name gets a Marathi fake and the text still reads naturally |
| Picking the actual fake name or address | Code, from local lists | Fast, consistent, and never repeats a real value |
| Fake ID numbers in the right format | Code (fakes.js) | Exact format, no AI needed |
| Saving the mapping | Code, chrome.storage.local | A model cannot store data. Code is simple and reliable |
| Putting real values back | Code, exact find and replace | Must be 100% exact. No guessing |
| The actual task (tables, summaries, letters) | ChatGPT | The big model does the hard thinking, but only on fake data |

### How each type is replaced

| Type | Found by | Replaced with | Example |
|---|---|---|---|
| Person name | Gemma | Realistic fake of the same language and gender | Ramesh Kumar → Suresh Patil |
| Address | Gemma | Fake address from a local list | 12 MG Road, Pune → 45 Station Road, Nashik |
| Aadhaar | Regex + Verhoeff | Fake 12-digit number, same spacing | 2345 6789 0123 → 8812 3409 7756 |
| PAN | Regex | Fake in format AAAAA9999A | ABCDE1234F → KLMPQ6621R |
| IFSC | Regex | Fake in format AAAA0XXXXXX | SBIN0001234 → HDFC0007812 |
| Phone | Regex | Fake 10-digit starting 6 to 9, same prefix style | +91 98765 43210 → +91 91234 56780 |
| Email / UPI ID | Regex | Fake handle, same shape | ramesh@okaxis → suresh@okaxis |
| Bank account, PIN, DOB | Regex (only near words like A/c, PIN, DOB) | Fake of the same length or format | A/c 001234567890 → A/c 004471209385 |
| Item added by user | User | Fake of a similar kind if the type is known, else tag [CUSTOM_1] | |

Rules: the same real value always gets the same fake inside one chat. A fake is never equal to any real value in the text. If the user picks 'Tags' in settings, every item uses tags like [NAME_1] instead of fakes. Amounts, salaries, prices and medical details are left as they are, on purpose.

## 5. Tech stack, part by part

| Layer | Technology | What it does in MaskIN | License |
|---|---|---|---|
| Browser | Google Chrome (desktop, macOS) | Where the extension runs. Safari is not supported | n/a |
| Extension format | Chrome Extension, Manifest V3 | The current standard for Chrome extensions | n/a |
| Language | Vanilla JavaScript, HTML, CSS | No framework, no build step. The folder loads directly with 'Load unpacked' | n/a |
| Page integration | Content script on https://chatgpt.com/* | Mask button, Send guard, Review popup, restore, badge | Our code, MIT |
| Popup isolation | Shadow DOM | Our popups use their own styles, so ChatGPT's CSS cannot break them and ours cannot break ChatGPT | Web standard |
| Brain of the extension | Background service worker (ES module) | Runs detection, calls Ollama, builds fakes, reads and writes the mapping | Our code, MIT |
| Side panel | Chrome sidePanel API | Status, settings, PDF drop, this chat's items, clear data | Our code, MIT |
| Local AI engine | Ollama (macOS app) | Runs the model on the Mac and listens at localhost:11434 | MIT |
| Local AI model | Gemma 4 E4B (default), E2B (8 GB Macs) | Finds names and addresses, and the style of each name (language, gender) | Open-weight. Exact license quoted from Google's model card in the README |
| Model API call | POST http://localhost:11434 /api/chat, stream false, format json, temperature 0 | Returns a clean JSON list of findings | n/a |
| Pattern detection | Regex + Verhoeff checksum (detectors.js) | Indian IDs and numbers. Devanagari digits are converted to normal digits first | Our code, MIT |
| Fake data | fakes.js + names.js (local lists of Indian names, surnames, streets, cities in English and Devanagari) | Realistic, consistent fakes | Our code, MIT |
| Storage | chrome.storage.local | Mapping per chat and settings. Stays on this laptop | Chrome API |
| Watching replies | MutationObserver | Notices new and finished replies | Web standard |
| PDF reading | pdf.js (pdfjs-dist), files bundled inside the extension | Extracts text from PDFs, and draws scanned pages as images for OCR | Apache 2.0 |
| OCR | Tesseract.js with English, Hindi, Marathi language files bundled inside the extension | Reads text from scanned PDFs and photos, fully offline | Apache 2.0 |
| Tests | Node.js scripts (node tests/run.js) | Checks every regex and the checksum with true and false examples | Our code, MIT |
| Built with | Claude Code and Antigravity (AI coding assistants) | Used to write code. Not part of the product. Named in the README | n/a |

## Manifest (key parts)

```
{
  "manifest_version": 3,
  "name": "MaskIN",
  "permissions": ["storage", "sidePanel"],
  "host_permissions": ["https://chatgpt.com/*", "http://localhost:11434/*"],
  "background": { "service_worker": "background.js", "type": "module" },
  "content_scripts": [{ "matches": ["https://chatgpt.com/*"],
      "js": ["config.js", "detectors.js", "fakes.js", "names.js", "content.js"],
      "css": [] }],
  "side_panel": { "default_path": "sidepanel.html" },
  "content_security_policy": { "extension_pages":
      "script-src 'self' 'wasm-unsafe-eval'; object-src 'self'" },
  "commands": { "mask": { "suggested_key": { "default": "Alt+Shift+M" } } }
}
```

Permissions are kept minimal. The extension can only touch chatgpt.com and the local Ollama address. No other website, no remote code. The CSP line lets the bundled OCR engine (WebAssembly) run, and still allows only our own files.

## What Gemma is asked (summary of the prompt)

System instruction: find names of people and addresses in Indian text (English, Hindi, Marathi). Return only JSON. Do not return ID numbers (regex handles those), amounts or medical details. Return exact text as it appears.

```
{ "items": [
  { "text": "Ramesh Kumar", "type": "name", "lang": "en", "gender": "male",
    "reason": "Full name of a person" },
  { "text": "12, MG Road, Pune 411001", "type": "address", "reason": "Home address" } ] }
```

Safety on the model output: an item is kept only if its text is actually found in the input. Anything else is dropped. If the JSON is broken, MaskIN retries once, then falls back to regex-only for that chunk.

## 6. Data and privacy: what stays, what leaves

| Data | Where it lives | Leaves the laptop? |
|---|---|---|
| Original text you type or paste | ChatGPT's input box, until you click Mask | No (it is replaced before sending) |
| Text sent to Gemma for detection | Ollama on localhost:11434 | No. Localhost means the computer talking to itself |
| PDFs, scans, photos | Read in the side panel by pdf.js and Tesseract.js | No. Never uploaded. OCR language files are bundled, nothing is downloaded at run time |
| Mapping (fake to real) | chrome.storage.local, one key per chat | No. We never use chrome.storage.sync, which would upload to Google |
| Masked text | Sent by the user to ChatGPT | Yes. This is the only thing that leaves, and it has fakes only |
| Restored replies | Only in the page on your screen | No. ChatGPT's servers still hold the fake version |
| Analytics, logs, tracking | None | No. MaskIN makes no network calls except to localhost |

**How chats are kept apart.** ChatGPT's URL has the chat ID (chatgpt.com/c/<id>). The mapping is saved under that ID, so follow-up questions in the same chat still restore correctly. A brand new chat has no ID until the first message, so the mapping is saved under a temporary key and moved to the real ID as soon as the URL changes.

**Honest limit.** chrome.storage.local is not encrypted. Someone with access to the unlocked laptop and Chrome could read it. Encrypting it with a passcode is our first stretch goal. The 'Clear' buttons let the user delete everything at any time.

## 7. Files and folder structure

```
maskin/
  manifest.json      the extension's ID card: name, permissions, files
  config.js          ChatGPT selectors (input box, send button, messages) and settings defaults
  detectors.js       all regex rules, Verhoeff checksum, Devanagari digit fix
  fakes.js           builds fakes for every type, keeps them consistent
  names.js           local lists: first names, surnames, streets, cities
  content.js         Mask button, Send guard, Review popup, restore, badge, Copy real
  content.css        styles for the Mask button only (popups live in Shadow DOM)
  background.js      pipeline: regex + Gemma, merge, fakes, mapping, Ollama status
  ollama.js          the only file that talks to localhost:11434
  storage.js         read, write, migrate and clear mappings
  sidepanel.html / sidepanel.js / sidepanel.css   status, settings, PDF, this chat, clear
  lib/pdf.min.mjs, lib/pdf.worker.min.mjs          pdf.js, bundled
  lib/tesseract/     tesseract.min.js, worker, core (wasm), eng/hin/mar language files
  icons/             16, 48, 128 px icons
  samples/           fake documents for the 8 use cases (text PDFs, scans, photos)
  tests/run.js       detector tests
  README.md, LICENSE (MIT)
```

**One rule for the code:** every ChatGPT selector lives only in config.js. When ChatGPT changes its website, only that file changes.

# 8. Setup on a Mac

1. Install Ollama from ollama.com (macOS app) and open it once.
2. Download the model in Terminal: `ollama pull gemma4:e4b` (on an 8 GB Mac: `ollama pull gemma4:e2b`).
3. Let the extension talk to Ollama. Without this, Ollama blocks the extension (this is called CORS):

```
launchctl setenv OLLAMA_ORIGINS "chrome-extension://*"
# then quit Ollama from the menu bar and open it again
```

4. Test it: `ollama run gemma4:e4b "List the person names in: Ramesh Kumar paid Sunita Patil"`
5. Open Chrome, go to `chrome://extensions`, turn on Developer mode, click **Load unpacked**, choose the maskin folder.
6. Pin the MaskIN icon, open chatgpt.com, open the side panel and check the Status card is green.
7. After any code change: press the reload icon on chrome://extensions, then refresh the ChatGPT tab.

Model choice: E4B on Macs with 16 GB RAM or more, E2B on 8 GB. Both run on Apple chips through Ollama. The choice is a setting, not a code change.

# 9. Use cases we demo and testing

We pitch MaskIN with these 8 real tasks. Every one uses fake sample documents only, never a real person's card.

| # | User's document | What they ask ChatGPT | What MaskIN hides |
|---|---|---|---|
| 1 | 10 KYC letters (text PDF) | Make a table of all names, addresses and phone numbers | Names, addresses, phones, Aadhaar |
| 2 | Salary slip | Calculate my tax under the new regime | Name, PAN, account number, employer address. Salary stays visible for the maths |
| 3 | Rent agreement | Summarise it and list anything unfair to the tenant | Landlord and tenant names, addresses, Aadhaar, PAN |
| 4 | Bank statement | Explain these charges and total my UPI spends | Name, account number, IFSC, UPI IDs. Amounts stay visible |
| 5 | Failed transaction details | Write a complaint email to my bank | Name, account number, UPI ID, phone, email |
| 6 | Photo of a filled form (fake sample) | Turn this into a clean typed summary | Read by OCR, then names, IDs, address hidden |
| 7 | Scanned Hindi or Marathi letter | Translate and explain it in English | Devanagari names and address |
| 8 | Hospital bill for an insurance claim | Check if the claim amount is correct | Patient name, address, phone, policy holder name. Bill amounts stay visible |

## Testing

**Pattern tests** (tests/run.js): for every ID type, values that must match and values that must not. A valid Aadhaar passes, the same number with one digit changed fails the checksum, a 12-digit order ID is not flagged.

**End-to-end checklist** before the demo: all 8 use cases run start to finish; follow-up question in the same chat restores; new chat works; Send guard fires; text PDF, scanned PDF and photo all work; Ollama off shows regex-only mode; Clear this chat works.

# 10. Build plan and team roles

**How we build:** the full first version is generated on one Mac (Shri's) with both Claude Code and Antigravity, each in its own separate folder from the same prompt. We test both, keep the better one, and push only that to the repo. Then each member improves their own part from their own laptop, with their own GitHub account, so every commit is real work they can explain.

| Time | Goal |
|---|---|
| 9:30 to 11:00 PM | Generate both versions. Pick the winner. First commit. Ollama + Gemma working on the demo Mac |
| 11:00 PM to 1:00 AM | Core flow solid: Mask, Review, swap, Send guard, restore, badge, per-chat mapping |
| 1:00 to 2:30 AM | PDF and OCR, side panel polish, Hindi/Marathi tests, run all 8 use cases |
| 2:30 to 3:30 AM | Bug fixes, README, backup demo video |
| By 4:00 AM | Final push. Submit on OrganizerHQ (Main Track). Add all members to the submission |
| 7:00 to 8:00 AM | Two full demo runs and pitch rehearsal |

| Member | Background | Owns | Pushes |
|---|---|---|---|
| Shri | Idea and prompting lead | Idea, prompts, first full build, testing the 8 use cases, pitch | First commit, README.md, samples/ (use cases 1 to 4) |
| Dev 1 | Web development | ChatGPT page: Mask button, Review popup, Send guard, restore, badge, selectors | content.js, config.js, content.css |
| Dev 2 | Web development | Side panel: status, settings, PDF and OCR, this chat list, clear data | sidepanel.*, storage.js, lib/ |
| Member 4 | Competitive programming | Pattern rules for Indian IDs, Verhoeff checksum, fake ID formats, tests | detectors.js, fakes.js, tests/, samples/ (use cases 5 to 8) |

Git rules: each member sets git user.name and user.email to their own GitHub account; the repo owner adds them as collaborators; always `git pull --rebase` before `git push`; members mostly edit their own files. Everyone runs the end-to-end checklist on their own Mac at least once.

# 11. Demo script and pitch (4 minutes)

| Time | What we say or show |
|---|---|
| 0:00 to 0:30 | Problem: people paste Aadhaar, PAN and bank details into ChatGPT every day. PS #3 asks us to stop that without stopping them from using ChatGPT |
| 0:30 to 2:30 | Live demo on chatgpt.com: paste a fake KYC letter with Hindi and English names → click Mask → Review popup with reasons → Apply → Send → ask 'make a table of names, addresses and phone numbers' → the table appears with real values and the '12 restored' badge → click 'Show what ChatGPT saw' to prove it only got fakes → press Enter on an unmasked message to show the Send guard |
| 2:30 to 3:30 | How it works: the 'who does what' table. Regex for exact formats, Gemma 4 on the Mac for names and context, code for mapping and restore. Show the side panel status: local AI connected |
| 3:30 to 4:00 | Quick flash of 2 more use cases (salary slip tax, scanned Marathi letter via OCR). Why Gemma. What we learned. Next steps: more chatbots (Claude, Gemini), encrypted mapping |

Backup: if the live demo fails, play the recorded demo video and keep talking over it.

# 12. Judge questions and answers

**Why not just use Gemma directly instead of ChatGPT?**

A small local model is much weaker than ChatGPT at real work like long documents, tables and analysis. So the small model does the private but simple job (spotting and swapping), and the big model does the hard job on safe data. And people keep using the chatbot they already trust.

**Does anything leave the laptop?**

Only the masked text that the user sends to ChatGPT. Detection runs on localhost, PDFs are read locally, the mapping is in local storage, and restore happens in the page.

**Why an extension and not a website?**

A website means copy, paste, copy, paste. Nobody keeps doing that. An extension works where the user already is, like Grammarly.

**What if Gemma misses a name?**

The Review popup shows everything before anything leaves, and the user can add missed items with one click.

**Why fakes and not [REDACTED]?**

ChatGPT works better on natural text. Sorting by name or drafting a letter still makes sense.

**What if ChatGPT changes a fake value?**

The reply is checked with the same detectors. Any ID-like value we did not create turns the badge amber so the user checks it.

**Why regex for IDs and not AI?**

IDs have fixed formats and must be exact. Regex is instant and never guesses. The Verhoeff checksum makes Aadhaar detection reliable.

**Why Gemma 4?**

It is open-weight, runs well on a MacBook through Ollama, understands Hindi and Marathi, and the E4B size runs fast enough on a MacBook Air for a live demo.

**Is Gemma truly open source?**

It is an open-weight model: anyone can download and run the weights. The slide asks for open-weight models. We quote its exact license in the README.

**Can the extension read my other websites?**

No. It only runs on chatgpt.com and only talks to localhost:11434. You can see that in manifest.json.

**What if ChatGPT changes its website?**

All selectors are in one file, config.js. It is a quick fix.

**Where is the mapping, and is it safe?**

In chrome.storage.local on this laptop, per chat. It is not encrypted yet, which is our first next step. The user can clear it any time.

**Why don't you hide salary or medical details?**

They don't point to a person by themselves. Once the name and IDs are swapped, those numbers belong to nobody, and ChatGPT needs them to do the calculation the user asked for.

**How does OCR stay private?**

Tesseract.js and its English, Hindi and Marathi language files are packed inside the extension. Scans and photos are read on the laptop, nothing is uploaded or downloaded.

**Did you use AI to code?**

Yes, Claude Code and Antigravity, and the README says so. Every part was tested by us and each member can explain their files.

# 13. Known limits (said honestly)

- Works on chatgpt.com in Google Chrome only, in this version.
- OCR is slower (a few seconds per page) and depends on scan quality. Limits: 20 pages for text PDFs, 10 for scans. The Review step catches what OCR misreads.
- Files that ChatGPT creates for download (like an Excel file) still contain fakes. Restore works on text and tables shown in the chat.
- A small model can miss things. That is why the user reviews before sending.
- Restore is on screen only. ChatGPT's own copy button copies the fake version; our 'Copy real' button gives the real one.
- The mapping is not encrypted yet.
- Gemma needs Ollama running. Without it, MaskIN falls back to regex-only mode and says so.

# 14. Glossary

| Term | Meaning |
|---|---|
| PII | Personally Identifiable Information: anything that points to a real person (name, Aadhaar, phone) |
| Chrome extension | A small add-on that runs inside Chrome, like Grammarly |
| Manifest V3 / manifest.json | The current rules for extensions / the file that declares name, permissions and files |
| Content script | Extension code that runs inside the ChatGPT page |
| Background service worker | Extension code that runs in the background and does the heavy work |
| Side panel | A panel on the side of Chrome that belongs to the extension |
| Shadow DOM | A sealed area of the page so our popup styles and ChatGPT's styles never mix |
| Ollama | A free app that runs AI models on your own computer |
| Gemma 4 E4B / E2B | Google's open-weight model, in a 4B and a 2B effective-size version |
| Open-weight | The model files are public. Anyone can download and run them |

| Term | Meaning |
|---|---|
| localhost:11434 | Your own computer, at Ollama's port. Nothing goes on the internet |
| CORS / OLLAMA_ORIGINS | A browser safety rule / the Ollama setting that allows our extension through it |
| API, JSON | A fixed way for two programs to talk / a neat structured text format for data |
| Regex | A pattern rule for text, for example 5 letters, 4 digits, 1 letter for PAN |
| Verhoeff checksum | The maths rule behind Aadhaar's last digit, used to confirm a number is a real Aadhaar format |
| Mapping | The small table of fake to real values, saved per chat |
| chrome.storage.local | Storage inside Chrome on this laptop only (sync would upload to Google) |
| MutationObserver | Browser feature that tells our code when the page changes, like a new reply |
| pdf.js | Mozilla's open library that reads text from PDFs inside the browser |
| OCR / Tesseract.js | Reading text from an image / the open library that does it inside the browser |
| WebAssembly (wasm) | Fast compiled code that runs in the browser. Tesseract.js uses it |
| MIT / Apache 2.0 | Open-source licenses that let anyone use, change and share the code |